Parameter Server

Developer quick reference · Single-node baseline · Python 3.14.4

Harness contract: use the public Python client. One real server communicates over gRPC; there are no REST endpoints or distributed consistency modes.

1. Connect and submit work

Import `Client` and `KeyRange` from `parameter_server`. Use an asyncio coroutine. `push` and `pull` copy inputs and return handles without awaiting network acknowledgement.

Client / handle operation	Purpose
`await Client.connect(address, worker_id, **options)`	Create a connected client.
`client.push(target, values, deps=(), iteration=0)`	Schedule numeric updates; returns `Task`.
`client.pull(target, deps=(), iteration=0)`	Schedule a snapshot; returns `Task`.
`await task.wait(timeout_ms=None)`	Return `Ack` for push or `Values` for pull.
`task.done()` / `task.task_id`	Local terminal-state check / operation identifier.
`await client.close()`	Close local activity; accepted server work is not rolled back.

2. Connection options and public attributes

Option	Default / meaning
`server_address`, `worker_id`	Required: `host:port` and unsigned 32-bit worker ID.
`sequence_start`	`0`; first unsigned 64-bit task sequence number.
`connect_timeout_ms`	`5000`; positive connection RPC timeout.
`submission_timeout_ms`	`5000`; positive registration RPC timeout.
`max_message_bytes`	`67108864` (64 MiB), send and receive.

Public attributes: `client.worker_id`, `client.next_sequence_number`, and `client.server_info`. Treat the worker ID as fixed. Reconnect with a new worker ID or a sequence above every previously issued ID, including uncertain requests.

3. Targets, values, and ordering

- **Targets:** `[2, 7, 19]` must be sorted and unique. `KeyRange(2, 5)` selects 2, 3, 4. Empty targets are valid.
- **Key space:** `0` to `2**64 - 1`. Select the last key with a sparse list or `KeyRange(2**64 - 1, 2**64)`.
- **Updates:** finite, flat, key-major numbers; length = key count × width. SGD applies `weights -= learning_rate * updates`.
- **Results:** `Values.keys` is an ascending tuple; `Values.data` is a local float64 array; `Values.width` is vector width. Reshape with `data.reshape(len(keys), width)`. `Ack.task_id` identifies a push; `TaskId` contains `worker_id` and `sequence_number`.
- **Dependencies:** earlier handles from this same client. The server waits for success; failed prerequisites prevent dependent execution. `iteration` is unsigned 64-bit metadata only.
- **Waits:** a local wait timeout does not cancel work; wait again. Repeated waits reuse the result or terminal error. Wait for required results before closing.

4. Startup and custom rules

```
python -m parameter_server --config config/server.json --log-level INFO
```

Log levels: `DEBUG`, `INFO` (default), `WARNING`, `ERROR`. Programmatic startup: `await start_server(config)` from `parameter_server.server`; returned server exposes `.port` and async `.close()`.

Before startup, call `register_update_rule(name, function)` from `parameter_server.rules`. Signature: `function(weights, updates, parameters) → weights`; arrays have shape `(key_count, width)`. Built-ins: `sgd` with `learning_rate=0.1`; `add` with `scale=1.0`. Rule failures leave storage unchanged.

Server & Protocol Reference

Configuration, RPCs, extension points, and errors

5. Server configuration

Use JSON via `--config` or instantiate `ServerConfig` from `parameter_server.config`.

Variable	Default	Meaning
`address`	`127.0.0.1:50051`	Server bind address.
`width`	`1`	Fixed vector width, at least 1.
`initial_value`	`0.0`	Finite scalar for unstored vectors.
`update_rule`	`sgd`	Registered rule name.
`rule_parameters`	`{}`	Named finite numeric parameters.
`max_keys_per_request`	`1000000`	Also constrained by snapshot size.
`max_message_bytes`	`67108864`	gRPC send/receive limit.

Demo configuration: `config/server.json` uses bind address `0.0.0.0:50051`, width 2, initial value 10, and learning rate 0.1. Clients use `127.0.0.1:50051` on the host or `server:50051` inside Compose.

`client.server_info` exposes width, initial value, request limits, and rule name—not rule parameters. The harness must retain its configured learning rate to calculate expected results.

6. gRPC endpoints

All RPC paths use this service prefix:

```
/parameter_server.v1.ParameterServer/
```

RPC	Request → response	Behaviour
`SubmitPush`	`PushRequest → Ack`	Register an update, not completion.
`SubmitPull`	`PullRequest → Ack`	Register a snapshot task.
`WaitTask`	`TaskId → TaskResult`	Wait for a terminal outcome.
`GetTaskStatus`	`TaskId → TaskStatus`	Read current state and terminal error.
`GetServerInfo`	`Empty → ServerInfo`	Read server settings.

Request fields: `task_id`, `target`, `dependencies`, `iteration`; push also has `values`. **States:** `PENDING → RUNNING → SUCCEEDED / FAILED`. `TaskResult` carries an `ack`, `values`, or `error`.

Identical task-ID retries reuse the original task; conflicting contents fail. Raw dependencies must already be registered and belong to the same worker. Wire-level details are in `parameter_server/proto/parameter_server.proto`.

7. Errors

Import from `parameter_server`. All inherit `ParameterServerError` and expose `.code`.

Python exception	Code / cause
`InvalidArgumentError`	`INVALID_ARGUMENT (1)`: validation, limits, or conflicting ID.
`UnavailableError`	`UNAVAILABLE (2)`: server/transport unavailable or client closed.
`TaskTimeoutError`	`TIMEOUT (3)`: local wait or RPC deadline expired.
`DependencyFailedError`	`DEPENDENCY_FAILED (4)`: prerequisite failed.
`InternalError`	`INTERNAL (5)`: execution/rule or unexpected transport failure.

A local wait timeout is retryable by waiting again. A submission deadline is a terminal handle error with a potentially uncertain server outcome. Execution failures arrive through `TaskResult`; the client translates them to Python exceptions.

Scope: in-memory, single server, plaintext gRPC. State and deduplication last only for the current server process; retained task history grows with workload. No manager, replication, vector clocks, or consistency modes.